

Arduino as a PLC server

On The Line Design Challenge

SensoredHacker

May 4, 2026

Abstract

Programmable Logic Controllers (PLCs) are industry-standard devices designed primarily for controlling high-voltage and industrial systems. Think of them as proprietary, industrial-grade Arduinos. They are not general-purpose computers, and often rely on obscure or archaic programming methods. First introduced in 1969, many PLCs still implement variations of Relay Control Logic (RCL), a system of representing electrical control diagrams that dates back over 160 years.

PLCs are complex in their simplicity. While their core concepts remain consistent, OEMs implement a wide range of communication protocols to enable control and data exchange. Through investigation, clear similarities emerge across these protocols; however, a fully open standard that enables true interoperability between platforms does not yet exist.

This lack of interoperability reinforces vendor lock-in. Once a system is built around a specific manufacturer's ecosystem, future expansion becomes constrained by compatibility requirements. Even when more suitable or advanced components become available, the cost and complexity of redesigning the system discourages change. In practice, this often limits integration not only across different manufacturers, but even across product lines within the same OEM.

This project explores the development of interoperable control systems using Arduino and Arduino-compatible devices to interface with and control PLCs. The goal is to investigate practical pathways toward more flexible, open, and adaptable industrial control architectures.

Nomenclature

Every OEM uses different terminology for their communication protocols. Terms like DiSM, MiSM, and various "Maintenance" labels often describe systems that are functionally similar, even when branded differently. This is confusing. To avoid that, this document will refer to all such systems as **Maintenance Protocol**.

Registers: While micro-controllers traditionally use registers, this project targets interoperability across both computer and micro-controller platforms. For clarity and consistency, the terms **Registers** and **Addresses** may be used interchangeably where appropriate.

Endian: Refers to the order in which bytes are stored in memory. PLC OEMs may describe this using terms such as "byte packing," "data format," or "32-bit storage set-

tings.” In this document, all such concepts of byte ordering and data layout will be referred to as **Endian**.

Register Map: Where specific data is stored. These mappings change from brand to brand. We are implementing an IDEC implementation of memory mappings from D0009 to D9999 and M0000 to M9999. The mappings change depending on system in use. I have a lot of PLCs, but use IDEC most often.

Goals:

- Implement Maintenance Protocol as a server on an Arduino.
- Expand functionality to other protocols and devices, such as i2c, SPI, and Canbus.
- Send and receive Maintenance Protocol packets.
- Create an Arduino library for PLC controls

Hardware:

Element14's On The Line Design Challenge in providing me with an Arduino Uno Q, to build this project out on. The implementation end goal is to be compatible with all Arduino compatible devices, including ESP32 and Teensy. If I had not been nominated for this competition, I would be using those platforms instead.

System Requirements:

- Memory: ah, D000-D9999 as a 16bit register would take roughly 20KB, and that is just 1/2 of the implementation. Predicted memory use is around 42K, if I just write out all the registers as 16 bit u.int16.

I could get creative to bring this figure down, but MCU flash memory will likely be the most limiting factor.

here I have wrote a basic example where 1 register is configured:

Sketch uses 15300 bytes (6%) of program storage space. Maximum is 253952 bytes.
Global variables use 1761 bytes (21%) of dynamic memory, leaving 6431 bytes for local

Project Plan:

What I want to do is just write a big ol table of registers. But that will hog memory. What I will do is slightly more convoluted perhaps. The plan is to write the server in python3, on my desktop, to get the server concept operational. Then test it with Things like my HMI, to verify operations. While I am fairly certain on what requests look like, I do not have the Return code patterns memorized yet. The server will basically be responding with return requests.

How will I implement other busses unsupported by things like the PLC / HMI? The plan here is to implement a form of extended memory, and treat other devices like native

input and output. The libraries I have written so far do not have the faculties to provide Analog IO, or extended memory. No big deal, I was going to write those eventually any way.

So then Your PLC, or HMI, could request data from Arduino devices, and shields. The only thing we would need to do is return specific data types.

Project Status

Active Development. Multiple client projects built, including some on Arduino Compatible hardware clients.

How The protocol works:

The protocol request pattern is :

ENQ = ACK/NAK

DEVICE = FF/1:N #BYTE

CONTINUATION = 1: wait for more instructions, or 0: don't.

TYPE = Read / Write / Clear

Datatype: bits or registers basically

value=bytes varies based on datatype.

checksum=BCC, an XOR operation on the sum of the data being requested.

terminator = \0h

Then the server responds with:

ACK/NAK = 06h / 15h indicating success or error

DEVICE = Echo of the addressed device (FF / 1:N)

STATUS = Execution result code (00 = OK, nonzero = error)

DATA = Returned value; format depends on datatype (bit = 0/1, register = word/bytes)

CONTINUATION = 1: more data follows, 0: final response

CHECKSUM = BCC, XOR of all response bytes excluding ENQ and terminator

TERMINATOR = \0d (carriage return)

Native Implementation

The most frequent implementation of an HMI is as a client device, which gets its values from the PLC. Modern HMIs can do both client and server operations of course. While I do hate distributing binary blob data, or proprietary sources, for this project, it is somewhat unavoidable. You can test drive the Automation Organizer suite for 14 days, and download it from IDEC. <https://mktg.idec.com/en-us/ao-demo-request>. The Automation Organizer suite will allow you to write Native compiling PLC and HMI code with WindLDR and WindO/I-NV4 respectively.

Any other example I could provide would be derivative in the sense that the objective is hardware integration, not how to avoid using IDEC software. You can of course roll your own HMI interface, but this is a separate blog topic.

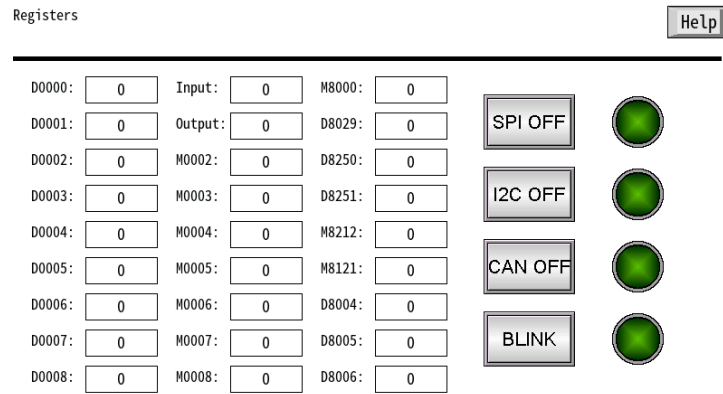


Figure 1: HMI Register Monitor

Caveats:

If your PLCs, and Arduino are all plugged into your Linux PC, you may have multiple devices named `/dev/ttyAMC*`. Make sure to write to the correct device.

Arduino Implementation examples

Test0:

Test0: basic operational testing. I built an Arduino server with a Mega, and the Ethernet R3 Shield. Then had my custom remote client read the server IP: 192.168.1.2 for a maintenance protocol packet.

Test1:

Configure the Arduino to present itself as a suitable host to the HMI

Reference Material:

The best document is a protocol guide: There are many documents on this subject, but the detail are often scattered, and implementation is not clearly defined. (I'll link more refs later)

Thanks!

Thank you to Element14, and the On The Line Design Challenge team in providing me with both hardware, and documentation, to build this project out on.

Legal

The FC6A.py library is open-source, as is Python3 and all the materials in this project. IDEC PLCs however, are proprietary. We do not assert any claim to software or hardware

licenses, or intellectual properties owned by IDEC. The `FC6A.py` library only provides a means to access and interpret resources already available to users of IDEC PLC systems. We assert that the use of this web page, and affiliated project files are for educational purposes only. Use at your own risk.

Ethical:

IDK, the OEM said I could do it, but they also said that this is not a topic they provide support for. Can PLCs be exploited? Does this project introduce new exploits to the PLC ecosystem? Yes, PLCs are vulnerable systems, with limited ability to internally manage secure operations. There are some strategies to manage this, the best is to keep your PLC systems off the internet. As for introducing new exploits, not really. This project and its applications are utilizing existing system behavior.

It is however possible that this project, or its derivatives may serve to impart important knowledge about PLC ecosystem security. There are Many PLC manufacturers, and they all have these issues. Knowing about the potential issues helps the community stay safe.

Security:

PLCs are not quite computers, and practically speaking, have no security. If you are using PLCs, it is our advice and sincere hope that you do so behind a firewall or VPN. Under no circumstance, give your PLC direct access to the internet. Yes, I know there are many features in integrations that point you to put these devices on the internet, and limited, basic example implementations of OEMs documenting doing just that, to send emails, connect to social media, run web servers, an other interesting IoT applications. Do it behind a firewall / vpn.

In the News:

PLCs are pretty boring honestly, but vulnerable systems do make the news periodically. Every PLC is a vulnerable system, and PLCs run all sorts of industrial processes. its not a new issue, PLC security is an on going issue. Exploits on PLC systems which worked 40 years ago work today, on vulnerable systems.

<https://www.cisa.gov/news-events/cybersecurity-advisories/aa26-097a>

<https://www.wired.com/2012/04/exploit-for-quantum-plc>

<https://en.wikipedia.org/wiki/Stuxnet>